embedded.c3l

API and Tooling Improvement Plan

Review against the c3c 0.8.6 pre-release (git cc5fafd3f)

6 November 2026

Abstract

embedded.c3l is a source-selected hardware abstraction layer (HAL) for the C3 language targeting the Arduino Uno (ATmega328P, elf-avr) and the ESP32-C3 SuperMini (elf-riscv32). This document reviews the library, its build configuration, its examples and its CI against the feature set of the current c3c compiler, and proposes a prioritised list of improvements. The goal is a library that a beginner can use within minutes (Arduino-style blink) while remaining suitable for professional firmware: explicit error handling, zero hidden cost, deterministic timing and a predictable build. Every claim about the compiler in this document was checked against the local c3c clone or by running the compiler.

Contents

1 Current state **2**
- 1.1 Repository layout . . . 2
- 1.2 What works today . . . 2
- 1.3 Compiler features relevant to this library . . . 2

2 Findings **3**
- 2.1 Build configuration and templates . . . 3
- 2.2 Public API . . . 5
- 2.3 Backends and runtime . . . 9
- 2.4 Examples, documentation and CI . . . 10

3 Tooling proposals **12**
- 3.1 Flashing through exec . . . 12
- 3.2 Size and map reporting . . . 12
- 3.3 Register definitions from vendor data . . . 12
- 3.4 Project scaffolding . . . 12
- 3.5 Editor support . . . 12

4 Upstream (c3c) items worth raising **13**

5 Prioritised roadmap **13**

1 Current state

1.1 Repository layout

Path	Role
`manifest.json`	Library manifest; two `templates` and two `targets`.
`src/api/gpio.c3i, time.c3i`	Public, backend-independent interfaces.
`src/compat/arduino.c3(i)`	Arduino-flavoured convenience shim on top of the API.
`src/core/core.c3`	`@volatile_load` / `@volatile_store` builtin macros, placed in `std::core::embedded`.
`src/boards/*/board.c3`	Per-board pin name constsets (`embedded::board`).
`src/backends/avr/atmega328p/`	GPIO, timer-0 clock, startup/vector table, compiler-rt shims.
`src/backends/esp32/esp32c3/`	GPIO, systimer clock, startup/watchdog disable, compiler-rt shims.
`linked-libs/elf-avr/arduino.ld`	AVR linker script (32 K flash, 2 K SRAM).
`linked-libs/elf-riscv32/esp32c3.ld`	ESP32-C3 linker script (IRAM/DRAM only).
`examples/blink, examples/digital_read, examples/watchdog`	Three examples, each with two project targets.
`.github/workflows/`	Example build and nightly zip release.
`flash.txt`	Hand-written flashing notes (Linux device paths).

1.2 What works today

Both examples build with the current compiler. Measured on this machine:

Target	Output	text	data	bss
`blink-arduino-uno`	19 KB ELF	4980	132	10
`blink-esp32c3`	12 KB ELF	–	–	–

The AVR build prints `WARNING! This architecture is unsupported.` The source of that warning is `arch_is_supported()` in `src/compiler/target.c`, which whitelists only wasm32/64, x86_64, aarch64, riscv32 and riscv64. AVR therefore has "best-effort" status in the compiler, which matters for the roadmap (Section 4).

1.3 Compiler features relevant to this library

The following `c3c` 0.8.6 features were confirmed in the local clone and are directly usable by `embedded.c3l`:

- **Library templates.** A manifest may define `templates`; a project target selects one with `"template": "lib/name"`. Template keys are validated against the full project-target property list, and target-local keys override the template (`project.c`, lines 918–932). Features from the template are *appended* to the target's features. Verified: templates may carry `extension`, `function-sections` and `data-sections`.
- **Section GC.** `function-sections` / `data-sections` (new in 0.8.5) place every function and global in its own section so `-gc-sections` can drop them.

- **Feature flags.** `@feat(A, B)` on declarations and modules, `$feat(X)` as a compile-time boolean, `-D/-U` on the command line and `features` in templates, projects and targets. Top-level `@if` is deprecated in favour of `@feat`.
- **Low-level attributes.** `@naked`, `@section`, `@noreturn`, `@export`, `@cname`, `@asmname` (0.8.6), `@weak`, `@nostrip`, `@used`, `@align`, `@noinline`, `@callconv` (only `cdecl`, `veccall`, `stdcall`; no interrupt convention).
- **Builtins.** `$$volatile_load/store`, the `$$atomic_*` family, `$$memcpy/memset(_inline)`, `$$trap`, `$$unreachable`.
- **Environment.** `std::core::env` exposes `FREESTANDING`, `FREESTANDING_ELF`, `NO_LIBC`, `ARCH_32_BIT`, `MEMORY_ENV`, `REGISTER_SIZE`, `COMPILER_OPT_LEVEL`, `SAFE_MODE`, etc.
- **Build pipeline.** `exec` scripts (shell or `.c3`) run per target, with `env`/`env-override` and `${NAME}` expansion (0.8.6), gated by `-trust=full`; `depends-on` between targets; `source-lib` target type and `c3c source-lib` to pack a `.c3l`; `-emit-asm`, `-emit-llvm`, `-print-linking`, `-stats`.
- **Manifest metadata.** `version`, `authors` and `langrev` are now accepted in `manifest.json`; `vendor` is a free-form extension slot ignored by the compiler.
- **Panic routing.** `-panicfn` names a function and feeds `$$PANIC_FN`; compiler-generated panics live in `std::core::runtime::panics`.

2 Findings

Findings are grouped by area. Each ends with a concrete proposal and a priority: **P1** (fix soon, affects correctness, size or every user), **P2** (meaningful quality-of-life or professional-use gap), **P3** (nice to have / long term).

2.1 Build configuration and templates

F1. Dead-code stripping is not enabled; firmware is more than twice as large as needed. The templates pass `-gc-sections` to the linker but do not set `function-sections`/`data-sections`, so the linker has nothing to collect. Adding both to the templates was tested with a copy of the library:

`blink-arduino-uno`	text	data	total
current templates	4980	132	5122
with `function-sections` + `data-sections`	2124	109	2243

A 57 % reduction with no source change. **Priority: P1** *Status: fixed, both templates now set `function-sections` and `data-sections`.*

F2. Boilerplate every user has to repeat. Each example target repeats `"type"`, `"name"` (equal to the key) and `"extension": ".elf"`. `extension` is a valid template key (verified), so it belongs in the template. `name` is redundant. A user target should be reducible to:

```
"blink-uno": { "type": "executable", "template": "embedded/arduino_uno" }
```

Priority: P1 *Status: fixed, both templates set `extension`, and the example targets are reduced to `type` and `template`. The output name defaults to the target key.*

F3. No safety, panic or debug policy in the templates. `Oz` implies "unsafe, no panic messages, no backtrace", which is fine for release but leaves no path to a debug build. Professional projects need two profiles per board. Proposal: ship two templates per board.

- `<board>` (release): `Oz`, `safe=false`, `panic-msg=false`, `debug-info=none`.
- `<board>_debug`: `O1` or `Os`, `safe=true`, `debug-info=line-tables`, `panic-msg=true`, `trap-on-wrap=true`.

Add a documented `panicfn` hook so that a panic can blink an LED or write to UART rather than spin silently. Since the templates merge with target-local settings, a user may still override any single key. **Priority: P2** *Status: fixed.* ***arduino_uno_debug*** *and* ***esp32c3_supermini_debug*** *build with* ***O1***, ***safe***, ***panic-msg***, ***debug-info=line-tables*** *and* ***panicfn = embedded::panic::handler***; *the release templates are unchanged (* ***Oz*** *already implies the proposed release settings).* ***O1*** *rather than* ***Os****: on the Uno it costs about 7 % more flash but 15 % less SRAM, because less inlining means fewer copies of the panic location strings, and SRAM is the scarce resource there.* ***trap-on-wrap*** *is left out: it applies to the library as well, whose runtime helpers and clocks wrap on purpose, and c3c has no per-expression opt-out. The new* ***embedded::panic*** *module (*`src/api/panic.c3`*) disables interrupts, calls a weak* ***embedded_on_panic*** *hook once (for putting outputs into a safe state), prints the message and location on* ***board::CONSOLE*** *(or a UART handed over with* ***panic::report_to****), then arms the watchdog and flashes the LED until the reset. The* ***professional*** *example now uses it instead of its own copy. Every example has* ***-debug*** *targets and CI builds them; all fit, the largest on the Uno is* ***professional*** *at 15.8 KB flash and 1.4 KB SRAM.* ***-Map*** *is not in the templates because a template cannot name a per-target file; a target can add* ***"link-args": ["-Map=build/x.map"]****. Not yet run on hardware.*

F4. One template per board bundles MCU and board; adding a second ATmega328P board would duplicate everything. The manifest currently binds `target` → one board. Because `targets` in the manifest are keyed by `arch-os`, two AVR boards would share the `elf-avr` target and therefore the same `sources` and linker script, which breaks as soon as a second AVR board appears. Proposal: keep MCU code selection in the manifest target (`elf-avr` compiles all AVR MCU backends, each module guarded by `@feat(MCU_ATMEGA328P)` etc.), move board pin maps behind `@feat(BOARD_...)` inside a single `src/boards/**` glob, and select the linker script through the template's `link-args` rather than the target's. Templates then stay thin: `cpu`, `features`, `link-args`. **Priority: P2** *Status: fixed. Every backend module carries* ***@feat(MCU_ATMEGA328P)*** *or* ***@feat(MCU_ESP32C3)*** *and every board module* ***@feat(BOARD_...)****. The manifest's top-level* ***sources*** *now hold everything shared (*`src/api`*,* `src/compat`*,* `src/core`*,* `src/rt` *and all of* `src/boards`*), each arch target adds only its backend directory (*`src/backends/avr/**`*,* `src/backends/esp32/**`*) and* ***-gc-sections****, and each template selects its linker script through* ***link-args****. The AVR script is renamed* ***atmega328p.ld*** *since it describes the MCU, not the Uno. A board file* ***$asserts*** *that its MCU feature is set, so pairing a board with the wrong MCU fails with a clear message. Checked by building all examples before and after: every section size and every symbol size is identical, only local-symbol suffixes differ. A throwaway second ATmega328P board (a copied board file plus a template) built next to the Uno without touching anything else.*

F5. Missing manifest metadata and schema. `manifest.json` has no `version`, `authors` or `langrev` (all accepted since 0.8.6), and no `vendor` block. A `vendor` block is the right place for machine-readable board data (flash tool, chip id, default port pattern, flash size) that a future flashing helper or IDE plugin can read without parsing C3. **Priority: P2** *Status: fixed.* ***manifest.json*** *has* ***version*** *(0.1.0),* ***authors*** *and* ***langrev****, and a* ***vendor.embedded*** *block (*`schema: 1`*) describing each MCU (arch, target, feature, linker script, flash/RAM/EEPROM sizes) and each board (template, MCU, feature, USB VID/PID, and flash settings: tool, image*

format, chip, programmer, baud, offset and flash mode where needed, and port patterns per OS). The flash settings are the ones from `flash.txt`, so an `exec` flashing helper (F7) can read them. A JSON schema for the vendor block is not written yet.

F6. `std::core::embedded` hijacks the standard-library namespace. `core.c3` declares `module std::core::embedded` so that `@volatile_load` and `@volatile_store` are `@builtin` and visible everywhere. It works, but any future stdlib module with that name collides, and users are surprised to find library code under `std::`. Proposal: `module embedded::core` with the same `@builtin` macros (`@builtin` does not require the `std::` prefix), and add typed register helpers (`@reg_read`, `@reg_write`, `@reg_set_bits`, `@reg_clear_bits`, `@reg_update`) so the five near-identical `reg_*` helper sets in the backends collapse into one. **Priority: P2** *Status: fixed, with one module left under `std::`. `src/core/core.c3` is `embedded::core` and holds `@volatile_load`, `@volatile_store`, `@compiler_barrier` and the new `@reg_read`, `@reg_write`, `@reg_set_bits`, `@reg_clear_bits` and `@reg_update`, which default to the MCU's register width (`char` on AVR, `uint` on the ESP32-C3) and take a type for others. They replace all seven helper sets (AVR `soc`, `gpio`, `wdt`; ESP32-C3 `soc`, `gpio`, `time`, `wdt`); `soc::read16/write16` stay because they encode the TEMP-register byte order. Two compiler facts differ from the proposal. `@builtin` outside `std::` only drops the module prefix, so every backend file now has `import embedded::core`. And c3c warns about every `$$` builtin used outside `std::` and `compiler_rt::`, once per macro expansion (about 180 warnings per build), with no way to silence it from a template. The five `$$` calls therefore sit in `std::core::embedded_intrinsics` (`src/core/intrinsics.c3`), a private shim with a name unlikely to collide; nothing else is under `std::`. All example images are byte-identical except `professional`, which builds with `safe` and is 830 B smaller on the Uno: the old helpers dereferenced `*(char*)addr` and so carried null checks, and the GPIO helper functions a shift-range check, none of which can fail. Host tests pass (they now include `src/core`).*

F7. `exec` is not used; flashing is a hand-copied command. `flash.txt` hard-codes Linux device paths and `esptool.py` invocations. The compiler can run post-build scripts per target (`exec`, `env`, `${NAME}` expansion) when the user opts in with `-trust=full`. Proposal: ship `scripts/` in the library with a portable `flash.c3` (or small shell scripts) and document the one-line opt-in; see Section 3. **Priority: P2** *Status: fixed, not yet run against a board. c3c runs `exec` before it compiles a target, not after, so the proposal in Section 3 would flash the previous build. Each example instead has `flash-arduino-uno` and `flash-esp32c3` targets of type `prepare` that `depends-on` the firmware target and run `scripts/flash.c3`, with `script-dir` pointing at the library's `scripts/` and `exec-dir` at the project. The script reads the board's flash settings from the manifest's `vendor` block, takes the port from `-port`, `PORT` or the single match of the board's pattern for the OS, calls avrdude (which reads the ELF directly, so the vendor format is now `elf`) or esptool (`elf2image`, then `write_flash`), and sends the tools' output to stderr so it appears live (c3c captures a script's stdout and prints at most 2 KB at the end). `FLASH_DRY_RUN=1` prints the commands; CI runs every flash target that way. Checked locally: dry runs, missing, ambiguous and single ports, a bogus port (avrdude's error fails the build), unknown boards and missing ELFs. `flash.txt` is replaced by `docs/flashing.md`. A failed script leaves c3c's `__c3exec__` binary behind, which is now in `.gitignore`.*

2.2 Public API

A1. Two incompatible `PinState` types. `embedded::compat::arduino` redefines `PinState` with the same values as `embedded::gpio::PinState`. As a result `digital_write` takes the compat type and `digital_read` returns the gpio type, so `digital_write(pin, digital_read(other))` does not type-check without a cast. Proposal: delete the duplicate and re-export with `alias PinState = gpio::PinState;` (or import the constants directly). Same for `HIGH`/`LOW`. **Priority: P1** *Status: fixed, the compat layer now aliases `gpio::PinState`.*

A2. `toggle` panics instead of returning the fault. Both backends implement `toggle` as `read_output(pin)!!` followed by `write(...)!!`. With `Oz` and no panic function this traps the MCU silently on an invalid pin, which contradicts the rest of the API that returns `INVALID_PIN`. Replace with the `!` rethrow operator. **Priority: P1** *Status: fixed, both backends now rethrow.*

A3. `Config` is positional and verbose at the call site. The compat layer has to write the following five times:

```
gpio::configure(pin, { Direction.OUTPUT, Pull.NONE, OutputType.PUSH_PULL,
                       DriveStrength.DEFAULT, PinState.LOW });
```

Proposal: order fields so zero-initialised defaults are sensible (`Direction.DISABLED`=0 is wrong for this; make `INPUT` the zero value), use designated initialisers in docs (`{ .direction = OUTPUT }`), and provide constant presets plus one-liners:

```
const Config OUTPUT_LOW   = { .direction = OUTPUT };
const Config INPUT_PULLUP = { .direction = INPUT, .pull = UP };

fn void? output(Pin pin, PinState initial = LOW);
fn void? input(Pin pin, Pull pull = NONE);
```

Default arguments are supported by C3 and remove most of the compat layer's reason to exist. **Priority: P1** *Status: fixed. `Direction.INPUT` was already the zero value. `gpio.c3i` documents designated initialisers and adds the presets `CONFIG_INPUT`, `CONFIG_INPUT_PULLUP`, `CONFIG_INPUT_PULLDOWN`, `CONFIG_OUTPUT_LOW` and `CONFIG_OUTPUT_HIGH` (prefixed so they cannot be confused with the `Direction` and `PinMode` members), plus `output(pin, initial = LOW)` and `input(pin, pull = NONE)`, which return the A4 handles. `pin_mode` now uses the presets.*

A4. No pin handle type; every call re-validates and re-decodes the pin. Each `write` on AVR recomputes port/bit via three range checks and calls `disable_pwm` (a six-way switch) on *every* write. For a bit-banged protocol this dominates. Proposal: add an `Output`/`Input` handle struct produced by `configure` that caches the decoded register address and mask, with methods `set`, `clear`, `toggle`, `read`:

```
Output led = gpio::output(board::BUILTIN_LED)!;
led.set(); led.clear(); led.toggle();
```

Keep the pin-number functions as the simple layer. The handle methods need no fault return, which also makes the hot path infallible and inlinable. **Priority: P1** *Status: fixed. Each backend defines `Output` and `Input`: AVR caches the PINx address and mask (2 bytes, PORTx is PINx+2, and `toggle` is a single PINx write), ESP32-C3 caches the mask. The methods are `set`, `clear`, `toggle`, `write`, `read` on `Output` and `read`, `is_high`, `is_low` on `Input`. They are listed only as comments in `gpio.c3i` because a `.c3i` cannot declare methods that a backend defines. With constant pins they compile to single `in/out` instructions on AVR. `gpio::write` no longer calls `disable_pwm`; `configure` and `reset` still do. `examples/digital_read` now uses the handles.*

A5. `Pin` is `ushort`, `board::Pin` is a constset. Fine today, but `supports()` and `is_valid()` are runtime checks on a value that is almost always a compile-time constant. Proposal: add `$assert`-based compile-time variants (`@output($pin)` macro) that reject an invalid pin at compile time when the argument is constant, and fall back to the runtime path otherwise. **Priority: P3**

A6. Time API: good shape, a few sharp edges.

- `Duration` is a 64-bit microsecond struct and `ms()` / `seconds()` go through a shift-and-add saturating multiply loop. On AVR a `ulong` multiply is already expensive; the saturating loop costs more code than a plain multiply with an overflow check (`@mul_overflow` /

`$$overflow_mul`). Also provide a `Duration32` or accept that `ms(ulong)` on a 16-bit MCU is heavy and document it.

- `clock_info()` reports resolution but the API has no way to ask for a *busy-wait vs. sleep* policy. Add `sleep` (may idle the core, feeds watchdog) and `spin` (pure busy-wait, interrupt-safe) as separate functions.
- `millis()` in the compat layer constructs an `Instant` with tick 0; document that boot time is tick 0 or expose `time::since_boot()` in the core API.
- The ESP32-C3 backend feeds the watchdog inside `has_elapsed`, i.e. inside the time API. Watchdog policy belongs in a `wdt` module, not in the clock. See A9.

Priority: P2 *Status: fixed.* `ms()` *and* `seconds()` *are a range check plus a plain multiply (the AVR runtime gained* `__muldi3`*, and both runtimes gained the 64-bit division and shift helpers); the cost on AVR is documented in* `time.c3i`*.* `spin(Duration)` *busy-waits without interrupts (a cycle-counted loop on AVR, systimer polling on ESP32-C3).* `sleep` *idles the AVR core between timer interrupts while the deadline is more than ~2 ms away and never feeds the watchdog.* `since_boot()` *is in the core API and the compat* `millis()/micros()` *use it. The watchdog item was already done under A9. The ESP32-C3 systimer clock enable used bit 30 instead of 29, which is fixed. The AVR clock was computed in 32 bits and wrapped after 71.6 minutes, which hung any deadline spanning the wrap; that is fixed too, and deadlines clamp very long durations instead of wrapping. No* `Duration32` *was added.*

A7. Interrupt model is absent from the public API, yet the backends rely on it. The AVR clock uses a hand-written `__vector_16` with 17 push/pop pairs, and the vector table is a 26-entry `rjmp` list. Users have no way to register a handler. Proposal:

- `embedded::irq` with `disable()`, `enable()`, `@critical { ... }` (save/restore SREG or mstatus) and typed `IrqLine` constsets per MCU.
- A weak-symbol vector table: every vector is `@weak @export("__vector_N")` defaulting to `__bad_interrupt`; a user or peripheral module overrides by defining the symbol. `@weak` and `@nostrip` are available today.
- Until an interrupt calling convention exists (Section 4), provide an `@isr(N)` macro that generates the naked prologue/epilogue so the user writes only the body.

Priority: P2 *Status: fixed.* `embedded::irq` *(src/api/irq.c3i) has* `enable`*,* `disable`*,* `is_enabled`*,* `save_and_disable/restore` *and* `@critical`*. String* `asm` *carries no memory clobber, so these use a new* `@compiler_barrier()`*. AVR: every vector jumps to* `__vector_N`*, unused ones are* `@weak` *defaults,* `__bad_interrupt` *is weak too, and* `irq::@isr("handler")` *is a five-instruction vector body that jumps to one shared register-saving trampoline, so handlers are plain exported functions. The time ISR uses it as well.* `@isr` *takes the handler's exported name because c3c has no way to get a function's link name. ESP32-C3: a vectored* `mtvec` *table, a common trap entry, and* `irq::attach(Source, handler, priority)` *which routes a peripheral source through the interrupt matrix; CPU exceptions call a weak* `embedded_esp32_exception`*. The ESP32-C3 start-up now clears* `.bss` *(it never did),* `.rodata` *moved to DRAM, and DRAM data now starts after the code, because IRAM and DRAM are the same physical SRAM and previously overlapped.*

A8. Missing peripherals. The README lists PWM, ADC, UART, I2C, SPI, external interrupts and clocks. UART existed in commit `f72ca45` (ESP32-C3 USB-Serial/JTAG, with a 200-line home-grown `printf`) and was removed in `02a0cba`. Proposed order, by how often a user needs it on day one:

1. **UART / console** – blocking `write`, non-blocking `try_read`, and an `io::OutStream` implementation so `std::io` formatting works once the stdlib is usable (see A10). Until then a minimal `print_uint`/`print_hex` is enough; do not re-implement `printf`.
2. **ADC** – `adc::read(Channel) -> ushort?`, with `Resolution` and reference selection in a `Config`.
3. **PWM** – `pwm::Channel` with `set_duty(permille)` and `set_frequency`. Note the AVR backend already knows which timer owns which pin (`disable_pwm`).
4. **External interrupts**, then **I2C**, **SPI** as blocking master drivers.

Each module should follow the same pattern: `.c3i` in `src/api`, `Capability` query, `Config` struct with defaults, fault set, and a handle type. **Priority: P2** *Status: implemented for both boards, compile-tested only (not yet run on hardware). New modules `uart` (blocking writes, non-blocking `try_read`, `write_uint`/`write_hex`; ESP32-C3 UART0/1 plus the USB-Serial-JTAG port as `board::CONSOLE`), `adc` (one-shot, handle with `read`/`read_millivolts`; ESP32-C3 uncalibrated), `pwm` (permille duty, per-channel frequency; AVR timers 0/1/2, ESP32-C3 LEDC), `exti` (AVR INT0/INT1, ESP32-C3 any GPIO), `i2c` and `spi` (blocking masters with timeouts; ESP32-C3 pins routed through the GPIO matrix). Each has a `.c3i` with `Config` (zero = defaults), a fault set and a handle type, plus a `supports()` query where pins differ. Examples: `uart_echo`, `analog_read`, `pwm_fade`, `button_irq`, `i2c_scan`, `spi_loopback`, all in the CI matrix. The `io::OutStream` part is not done because it needs the stdlib (A10).*

A9. Watchdog handling is implicit and scattered. The ESP32-C3 startup disables three watchdogs, and then GPIO and time functions still feed TIMG0 on every call (`feed_mwdt` in `gpio::write`, `read`, `configure`, `has_elapsed`). If the watchdog is disabled, feeding is wasted code and bus traffic in the hottest paths; if it is intended as a safety feature it must be explicit. Proposal: `embedded::wdt` with `disable()`, `enable(Duration)`, `feed()`; startup disables; nothing else touches it. **Priority: P1** *Status: fixed for `disable()` and `feed()`. `embedded::wdt` exists for both backends, startup calls `wdt::disable()` (the AVR startup now also clears a watchdog left running after a watchdog reset), and GPIO/time no longer feed anything. `enable(Duration)` now exists too. It rounds up to the nearest timeout the hardware supports and returns that timeout. The AVR backend uses* $16\,ms \cdot 2^n$*, the ESP32-C3 backend uses MWDT0 from XTAL with* $1\,\mu s$ *ticks. `info()`, `is_enabled()` and `caused_reset()` were added, along with an `examples/watchdog`. Two ESP32-C3 bugs were also fixed: the config-update bit is 22, not 13, and `feed()` has to unlock write protection.*

A10. The standard library is switched off entirely. `use-stdlib=false` means no `String` methods, no `io`, no `math`, no `List`, and the library must ship its own `memset`/`memcpy`/division shims. The stdlib has a `_nolibc` layer and `env::FREESTANDING`; the ESP32-C3 (RISC-V, 384 KB RAM) is a realistic target for `use-stdlib=true` with `link-libc=false` and `memory-env=tiny`. Proposal: investigate per board. Keep AVR freestanding; for RISC-V try the stdlib with a tiny bump allocator and report which modules link. This is the single biggest lever for "capable of professional projects". **Priority: P2** *Status: declined.*

A11. Compat layer naming. There are five names for two functions:

`delay`, `delay_ms`, `delay_us`, `delay_microseconds`, `delayMicroseconds`

Keep the two Arduino names `delay` and `delay_microseconds`, mark the camelCase one as `@deprecated` and drop the rest. The compat layer should be a thin, faithful Arduino surface, not a second API. **Priority: P3**

2.3 Backends and runtime

B1. Module naming is inconsistent. The AVR runtime module carries the full backend path, while the ESP32-C3 one is a plain `module runtime @private;`. Adopt one scheme everywhere:

```
module embedded::backends::<arch>::<mcu>::<unit>;   // MCU code
module embedded::boards::<board>;                   // pin maps
```

This lets two boards coexist in one build directory. **Priority: P2** *Status: fixed for the backends. Both runtime modules are gone (see B2), the private backend modules are `embedded::backends::<arch>::<mcu>::<unit>` (`startup`, `soc`, `regs`), and the shared runtime is `embedded::rt`. Backend files that implement a public interface keep its module name (`embedded::gpio` and so on), which is how a `.c3i` is implemented. The board module stays `embedded::board`: it is the selected board's public API (`board::Pin.D13`), only one board is compiled per target, and renaming it would break every user. Several boards in one build is F4.*

B2. Compiler-rt shims are duplicated and hand-rolled. `__mulsi3`, `__udivsi3`, `memset`, `memcpy` exist in both backends with different algorithms. Proposal: one `src/rt/` directory, included for every freestanding target, with `@weak @nostrip @cname("...")` definitions following the stdlib's own `_compiler_rt` style, and `@feat(ARCH_32_BIT)` guards where the width differs. Prefer `$$memcpy_inline`-based implementations where the backend is allowed to expand them. **Priority: P2** *Status: fixed. `src/rt/` (`embedded::rt`) is in both targets' sources: `mem.c3` (`memset`, `memcpy`, `memmove`), `int32.c3` (`__mulsi3`, 32-bit division), `int64.c3` (`__muldi3`, 64-bit division), `shift64.c3` (64-bit shifts, `@feat(!BACKEND_AVR)` because C `int` is 16 bits there) and `avr.c3` (`@feat(BACKEND_AVR)`: `__mulqi3`, `__mulhi3` and the libgcc-ABI `__[u]divmod{qi,hi,si}4`). Every helper is `@weak @export`. `@nostrip` is not used because it would defeat section GC, and the loops are plain because `$$memcpy_inline` needs a constant length. All helpers were checked on the host against native arithmetic, two million random and edge-case operand pairs each. Consolidating them found three bugs. The AVR `__[u]divmodhi4` and `__[u]divmodsi4` returned quotient and remainder in swapped halves, so every 16- and 32-bit `/` and `%` on AVR was wrong; the UART baud, I2C clock, PWM and ADC calculations use them. `%` on `char` did not link on AVR because `__mulqi3` was missing. The RV32 `__udivsi3` returned 0 when the dividend had its top bit set (latent, since the M extension divides in hardware). The ESP32-C3 also had no `memcpy`. 32-bit multiply code on AVR is 118 bytes smaller.*

B3. Magic numbers for registers. `0x2B`, `0x44`, `0xB0` on AVR and `(1u << 12) | (1u << 9)` on ESP32-C3 are unnamed. Proposal: a `regs.c3` per MCU with named constants (`PORTD`, `DDRD`, `IO_MUX_FUN_IE`, ...) and bitstructs for multi-field registers. C3 `bitstruct` with an explicit backing `uint` maps very well onto peripheral registers and documents the layout in code. Longer term, generate `regs.c3` from SVD/ATDF files (Section 3). **Priority: P2** *Status: fixed, except the SVD/ATDF generation. Each MCU has a `regs.c3` (`embedded::backends::<arch>::<mcu>::regs`) with every register address and bit field the backend uses, named after the datasheet or the ESP-IDF `soc` headers and grouped by peripheral: 133 constants for the ATmega328P and 291 for the ESP32-C3, including the GPIO-matrix signal numbers. The per-module copies and duplicates are gone (ESP32-C3 `gpio` and `time` redefined `soc`'s registers), and the remaining raw values (`gpio`'s port addresses and PWM-pin table on AVR, `(1u << 12) | (1u << 9)` and inline field masks on ESP32-C3) are named. Values that encode a choice (prescaler, PWM mode, ADC reference) stay in their module but are built from `regs::` fields. All 18 example images are byte-identical before and after. No bitstructs: c3c 0.8.6 does not accept a bitstruct literal cast to `uint` as a constant in another module, so the multi-field layouts are documented in comments. The `reg_*` helper duplication remains part of F6.*

B4. Linker scripts are minimal.

- AVR: no `__stack_top` symbol (stack is set from a literal in `__init`), no `.noinit`, no `ASSERT` on SRAM overflow, no `.eeprom`. `.rodata` is copied to SRAM (correct for classic AVR, but costs RAM; document it).
- ESP32-C3: no flash/XIP layout, so the image lives in IRAM only and must be loaded by the ROM loader into RAM; `.rodata` is placed in IRAM (works, but wastes instruction RAM). Add `__heap_start`/`__heap_end` for the future allocator.

Add `ASSERT(. <= ORIGIN(SRAM) + LENGTH(SRAM), "RAM overflow")`-style checks so an oversized build fails at link time rather than at runtime. **Priority: P2** *Status: fixed, except for the ESP32-C3 flash (XIP) layout. Both scripts define `__stack_top`, `__heap_start`/`__heap_end` and a `NOLOAD .noinit` section, reserve `__stack_size` (128 B on AVR, 8 KB on ESP32-C3, overridable with `-defsym`), and `ASSERT` that `.data`, `.bss` and `.noinit` leave it free. AVR: `.data` is placed `> SRAM AT> FLASH`, so lld also checks the flash image size; start-up copies from `LOADADDR(.data)` (`_sidata`) and takes SP from `__stack_top` instead of a literal. The copied `.rodata` is documented, and an `.eeprom` section sits at 0x810000. ESP32-C3: `.rodata` already moved to DRAM under A7. Running from flash needs an image header and MMU set-up from a second-stage bootloader, which is a project of its own.*

B5. Startup is correct but opaque. Both `__init` functions are `@naked` and mix inline asm with C3 statements (bss loop, calls). Mixing is fragile in a naked function because the compiler may spill. Proposal: keep `__init` asm-only (set SP, jump to `_start_c3`), and do data/bss init in a normal function. Also expose a `@init`-style hook list or a single `embedded::on_boot()` weak function for board bring-up. **Priority: P2** *Status: fixed. The AVR `__init` is asm only: it clears SREG and r1, sets SP, copies `.data` (which needs `lpm` and so cannot be C3) and jumps to `embedded_avr_start`, a normal function in `.init` that clears `.bss` and calls `wdt::boot`, the clock init, the hook and `start`. The ESP32-C3 start-up already had this shape. Both boards call a weak `embedded_on_boot` once the library is initialised and interrupts are on, just before `start()`. Firmware overrides it with `fn void ...() @export("embedded_on_boot")`, documented in the README.*

2.4 Examples, documentation and CI

E1. CI never builds the ESP32-C3 target. `example-build.yml` runs `c3c build` in `examples/blink` with no target, which builds only the first target (observed locally: only `blink-arduino-uno.elf` is produced). The matrix also lists only `blink`. Fix: build a matrix over example and target, and call the compiler with an explicit target name. **Priority: P1** *Status: fixed, the matrix is now example × board and passes the target explicitly.*

E2. CI uses the stock `c3c` release while the repo tracks the pre-release. The last commit is "making it work with the current compiler". Pin the compiler version used in CI (the `setup-c3` action input) and state the minimum version in the README and manifest `langrev`. Run the matrix against both the latest release and the nightly so breakage is noticed before users hit it. **Priority: P1** *Status: fixed. CI passes `version` to `setup-c3` explicitly and runs the matrix against `nightly` (required) and `stable` (`continue-on-error` until a stable 0.8.6 exists), printing the compiler version. The README states 0.8.6 as the minimum and the manifest sets `langrev`. A fixed pre-release tag cannot be pinned because c3c only publishes a moving `latest-prerelease-tag`.*

E3. Nightly release zips by hand. Replace the `zip` step with a `source-lib` project target (or `c3c source-lib`) so the compiler packages the library in the format `c3c fetch` expects, and attach a size report (`avr-size`, `riscv32-esp-elf-size`) of the examples to each release to track regressions. **Priority: P2** *Status: fixed for the size report, not possible for*

source-lib. c3c source-lib writes its own manifest with only provides, version, authors, langrev, dependencies and the source patterns, dropping templates, targets, vendor and linklib-dir, and it does not pack the linker scripts, so the result is not usable. The release keeps zip, which is the format c3c fetch expects (manifest at the root), now limited to the manifest, licence, README, src, linked-libs, scripts and docs/flashing.md (the examples are no longer in the archive). A new step builds a project against the archive and runs its flash target as a dry run through the unpacked copy. The size report comes from scripts/size.c3, which reads the ELF section headers itself (no avr-size or RISC-V binutils needed on the runner) and compares flash and RAM use with the MCU limits in the vendor block; it skips the ESP32-C3's .dram_dummy, which size counts as a second copy of the code. The report is attached to the nightly release, put in its notes, and each CI job adds its example's sizes to the job summary.

E4. Repository hygiene. No root `.gitignore`; the `.idea/` directory is committed; `flash.txt` sits at the root. Add `.gitignore` (`build/`, `.idea/`, `*.c3l` archives), move flashing docs to `docs/flashing.md`, and remove IDE files. **Priority: P3**

E5. Documentation. The README is a list of two commands. Needed: a 5-minute quick start (install, `c3c fetch` or submodule, write `project.json` with a template, build, flash), an API reference generated with `c3c docgen`, a "porting a new board" guide, and a page per board (pin map, what the template enables, flash procedure, known limitations). **Priority: P2**

E6. Examples. Add at least: `button_irq`, `uart_echo`, `adc_read`, `pwm_fade`, and a `professional/` example showing a non-blocking main loop with deadlines, fault handling and a `panicfn`. Use `@feat(BOARD_...)` for per-board constants as `digital_read` already does. **Priority: P2** *Status: fixed. button_irq, uart_echo, pwm_fade and analog_read (the ADC example) came with A8. The new professional example has a non-blocking main loop of deadline-driven tasks (heartbeat, debounced button poll, console commands, status report with worst-case loop time), keeps running when a peripheral fails to open, counts UART line errors, feeds the watchdog once per pass and reports a watchdog reset at boot. Its targets set safe, panic-msg, show-backtrace and panicfn. Without the stdlib, c3c calls the named function directly from its generated std::core::runtime::panics functions, and the handler prints the summary, file, line and function, then re-arms the watchdog and flashes the LED until it resets the chip. The checks cost about 2.9 KB of flash and 1 KB of SRAM on the Uno (AVR keeps the panic texts in RAM). Built for both boards and in the CI matrix, not yet run on hardware.*

E7. Tests. There are none. Two cheap layers: host-side unit tests for the pure parts (duration arithmetic, pin decoding tables, config validation) built with `c3c test` on the host target, and a "compile-only" CI job that type-checks every example for every template. **Priority: P2** *Status: fixed. The compile-only layer is the example × board CI matrix from E1. test/ is a host project with one test target per MCU (c3c test atmega328p, c3c test esp32c3), each compiling the real library sources it needs plus that MCU's GPIO backend. It has 39 and 34 tests: the runtime helpers against the host's arithmetic (exhaustive 8-bit and sampled 16-bit AVR divmod, random 32/64-bit operands, every 64-bit shift amount, memmove in both directions), duration and deadline maths against a fake clock (saturation, counter wrap, the 2^{62} deadline clamp), uart_format against a fake port (32-bit boundary, long.min, hex padding, read_byte timeout) and the GPIO pin tables (ATmega328P port decoding, ESP32-C3 valid pins and IO_MUX/GPIO register addresses). Putting back the old AVR quotient/remainder swap or removing the deadline clamp makes them fail. To make the time code testable, the duration and deadline functions both backends had copied moved into src/api/time_common.c3, the same way uart_format.c3 is shared. Its 64-by-32-bit divider is now @private, so it gets inlined: uart_echo is 16 bytes larger on AVR and 32 bytes smaller on ESP32-C3, and the other examples are unchanged. Compiling for the host also turned*

up two GPIO spots that only built because of the library's lenient `@volatile_store`: they now cast explicitly (AVR) and widen through `uptr` (ESP32-C3), with no change on target. CI runs both test targets.

3 Tooling proposals

3.1 Flashing through `exec`

(Correction: `c3c` runs `exec` entries before a target is compiled, so flashing has to be a `prepare` target that `depends-on` the firmware target; see F7.) `c3c` runs `exec` entries after a target is built. A template cannot force this on users (and should not, because it needs `-trust=full`), but the library can ship the scripts and the documentation:

```
"targets": {
  "blink-uno": {
    "type": "executable",
    "template": "embedded/arduino_uno",
    "exec": [ "flash-avr.sh build/blink-uno.elf ${PORT}" ]
  }
}
```

Run as

```
PORT=/dev/cu.usbmodem1101 c3c build blink-uno --trust=full
```

The AVR script converts to Intel HEX with `avr-objcopy` and calls `avrdude`; the ESP32-C3 script calls `esptool elf2image` and `write_flash`. Both should auto-detect the port when `PORT` is unset (`/dev/cu.usbmodem*` on macOS, `/dev/ttyACM*` on Linux, `COM*` on Windows). A `flash.c3` script is also possible, since `exec` compiles and runs `.c3` files, which keeps the toolchain single-language.

3.2 Size and map reporting

Add a `size` target or script that prints `text/data/bss` per build and fails above a threshold configured in `vendor` metadata (e.g. 32 K flash, 2 K SRAM for the Uno). Pass `-Map` via `link-args` in the debug template so users can inspect what pulled in what.

3.3 Register definitions from vendor data

Both vendors publish machine-readable register descriptions (Atmel `.atdf`, Espressif SVD). A small generator (Python or C3) producing `regs.c3` with constants and bitstructs would remove the magic numbers (B3) and make adding an MCU mostly mechanical. Place it under `tools/` and check in the generated output so users never run it.

3.4 Project scaffolding

`c3c init` accepts `-template <path>`. Ship `templates/project/` in the library so a new firmware project is one command:

```
c3c init myfw --template path/to/embedded.c3l/templates/project
```

The generated `project.json` should already contain one target per supported board using the library templates.

3.5 Editor support

Publish the `board::Pin` constsets and the API `.c3i` files as the documented surface and run `c3c docgen` in CI to a GitHub Pages site. The `-lsp` flag exists; recommend the C3 language server in the README with a snippet for the two IDEs most users have.

4 Upstream (c3c) items worth raising

These cannot be solved inside the library and are worth filing or contributing:

1. **Interrupt calling convention.** LLVM supports `avr-interrupt`/`avr-signal` and the RISC-V `interrupt` attribute. Exposing them as `@callconv("interrupt")` would delete the hand-written push/pop in `__vector_16` and make user ISRs safe.
2. **AVR as a supported architecture.** `arch_is_supported` excludes AVR, hence the warning on every build. A test-suite entry for `elf-avr` (compile-only) would be the first step; the library's examples are good candidates.
3. **`extension` and output naming in templates.** Works today but is undocumented; a documentation PR listing which keys templates may set would help library authors.
4. **Linker-script discovery.** `-T arduino.ld` resolves only because `linklib-dir` is added as `-L`. A first-class `linker-script` key would be clearer and would allow templates to switch scripts per board (F4).
5. **`c3c build` with no target builds only the first target.** Either document it or add `c3c build -all`.
6. **Freestanding stdlib subset.** A documented list of stdlib modules that compile with `use-stdlib=true`, `link-libc=false`, `memory-env=none` would unblock A10.

5 Prioritised roadmap

Step 1. Template hardening (one afternoon). Add `function-sections`, `data-sections`, `extension` to both templates; add `version`/`authors`/`langrev`; strip `name`/`extension` from the examples; fix CI to build every target of every example and pin the compiler version. (F1, F2, F5, E1, E2)

Step 2. API correctness. Unify `PinState`; make `toggle` rethrow; move watchdog handling into `embedded::wdt`; add `gpio::output`/`gpio::input` one-liners with default arguments. (A1, A2, A3, A9)

Step 3. Handles and hot paths. Introduce `Output`/`Input` handles with cached register and mask; stop calling `disable_pwm` on every write. (A4)

Step 4. Debug profile and panics. Add `<board>_debug` templates and a documented `panicfn`; add `ASSERT`s to linker scripts. (F3, B4)

Step 5. Restructure backends. Consistent module names, shared `src/rt`, named registers, asm-only `__init`. (B1, B2, B3, B5)

Step 6. Interrupts and UART. `embedded::irq`, weak vector table, `@isr` macro, then UART with an `OutStream`. Re-add the `uart_echo` example. (A7, A8)

Step 7. Tooling. `exec`-based flashing scripts, size report in CI, `source-lib` packaging, `c3c init` project template, generated docs. (F7, E3, Section 3)

Step 8. Stdlib on RISC-V. Evaluate `use-stdlib=true` for the ESP32-C3 template; add a bump allocator; document what works. (A10)

Step 9. Peripherals. ADC, PWM, external interrupts, I2C, SPI, each with the same `.c3i` / Config / Capability / handle pattern. (A8)

Step 10. Upstream. File the interrupt-callconv and AVR-support issues with the examples as reproducers. (Section 4)

Steps 1 and 2 need no API discussion and halve the firmware size while removing the two correctness traps. Steps 3 to 5 define the “professional” shape of the API and should be done before more peripherals are added, because every new module will copy the pattern that exists at that time.